CSC1202, 2026 – Week 3: Prompt Engineering: Foundations and Techniques

- **Part 1:** Prompting and Prompt Engineering
- **Part 2:** Prompting Techniques
- **Part 3:** System Prompt vs User Prompt
- **Part 4:** Structuring prompts

References: REF1: J. Alammar and M. Grootendorst, *Hands-On Large Language Models*, O'Reilly, 2024. **REF2:** Anthropic, *Prompting Best Practices*, Claude Documentation. **REF3:** Google, *Prompt Design Strategies*, Gemini API Documentation. **REF4:** OpenAI, *Prompt Engineering*, OpenAI API Documentation. Additional references are provided on the relevant slides.

Part 1: Prompts and Prompt Engineering

What is a Prompt

- A **prompt** is the input given to a generative model (e.g., an LLM) to obtain a desired output. It can include instructions and other information that guides the model's response.
- Prompting is the main way to **instruct and interact** with an LLM
    - In a chat interface: the message the user types (e.g., ChatGPT, Claude)
    - In an application: the text sent through an API call, possibly filled from a template

- The model only has access to the **information and instructions provided in the prompt** — it cannot know what the user intended but did not specify
    - ○ e.g., a lecturer provides a **research article** and asks "summarise this," while actually wanting **3 simple bullet points explaining the main findings to first-year students** → the model cannot know these unstated requirements
- A prompt can combine **task instructions, context, input data and output guidance** (more on prompt structure later)

Why prompting matters (1)

- LLMs are **general-purpose models**: the same model can summarise a lecture, translate an email, write SQL or answer a question about Loop
- The model is not tied to a single task: the **prompt** specifies the task and shapes the answer, without retraining the model
- **Different ways of prompting the model for the same task can lead to very different results**
  - e.g., changing only the **formatting of a few-shot prompt**—such as separators, spaces, or capitalization—can lead to **substantial differences in model performance** (Sclar et al., ICLR 2024).

Ref: M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying About Prompt Formatting," *ICLR*, 2024.

Why prompting matters (2)

- For most users, the prompt is the main **interface with the model**
- In applications, prompts are how developers **guide and control model behaviour**, so poorly designed prompts can lead to **incorrect or unreliable** application/agent behaviour
- Hence the need to **design, test and refine prompts systematically**, rather than relying on trial and error: **prompt engineering**

Example: Prompt and Model Output

Prompt

Generate PostgreSQL code given the following tables:

Table: schools
Columns: SchoolId, SchoolName

Table: students
Columns: SchoolId, StudentId, StudentName

Task: Write a SQL query to list all students in the “School of Computing”.

Example model output

```
SELECT StudentId, StudentName

FROM students

WHERE SchoolId IN (
    SELECT SchoolId
    FROM schools
    WHERE SchoolName = 'School
of
Computing'
);
```

Programming with prompts (1)

- **Traditional programming**
  - Behaviour is specified in a formal language (e.g., Python, SQL) with defined syntax and semantics
  - Invalid syntax is rejected; instructions must conform to the rules of the programming language
  - Deterministic code, given the same input and state, produces the same output

- **Prompting**
  - Behaviour is specified using natural language: instructions, context and examples
  - Ambiguity may not be rejected — **the model can proceed by interpreting an underspecified or ambiguous prompt**
  - The same prompt can produce **different outputs across different runs**, even without changing the prompt (**more on why later**)

Programming with prompts (2)

- Prompting can therefore be viewed as an **emerging way of programming models**: desired behaviour is specified through natural-language instructions rather than entirely through traditional code — related to Karpathy's **"Software 3.0"** (see lecture 1 slides)

- In applications, prompts can be **managed like code**: stored as templates, versioned, tested and reused **(e.g., a prompt template includes a {document} placeholder; when the prompt is used, {document} is replaced with the actual document to be processed)**

Ref: A. Karpathy, Software Is Changing (Again), keynote, Y Combinator AI Startup School, June 2025

Prompt engineering

Prompt engineering is the process of designing, testing and refining prompts so that a model **reliably produces the desired output**

- In prompt engineering, the **model itself is kept fixed** and the prompt is what we change — we change the input rather than the model's internal weights (**more on fine-tuning later**)
- **Task description**: specify what is to be done and relevant constraints or assumptions
  - e.g., "use only the tables given below", as in the SQL example
- **Structuring inputs**: clearly separate instructions from the data they apply to
  - e.g., using markers/delimiters or XML tags
- **Choosing a prompting technique**: e.g., **zero-shot, few-shot, chain-of-thought, ReAct,** ..
- **Iterative process**: design → test → evaluate → refine, similar to software development

Prompt engineering does not guarantee correct outputs: better prompts can reduce failures, but cannot eliminate them

Prompt Engineering as an Iterative Process

- A prompt is **not written once**: the first version is a hypothesis about how the model will respond
- **Design**: write a first version and choose an appropriate prompting technique (e.g., **zero-shot, few-shot**)
- **Test**: run the prompt on a **representative set of inputs**, not just one, and collect the outputs
  - e.g., suppose the task is to classify student feedback into **Teaching, Labs or Assessment**: test the prompt using multiple feedback examples covering all three categories
- **Evaluate**: check the collected outputs against **defined acceptance criteria** to determine whether the prompt performs well enough
  - at least **90% of the feedback must be classified correctly**, and every output must use **exactly one of the three predefined categories—no other category is allowed**.
- **Refine**: identify the cause of failures, make targeted changes to the prompt, then re-run the same test set
- The cycle aims to produce a prompt that **fails less often on the tested cases**, not a universally "correct" prompt

Prompt engineering cycle: Example (1) — Design

Task: Classify each GitHub issue as **BUG, FEATURE or QUESTION**

Design — **Prompt v1**:

> "What is this issue about?
> <issue>{issue}</issue>"

- **{issue}** is a placeholder replaced by the actual GitHub issue text
- **<issue> … </issue>** marks where the issue text begins and ends

- **Prompt v1 is the initial design; the next step is to test how it performs on different GitHub issues**

Prompt engineering cycle: Example (2) — Test & Evaluate

Test: Run Prompt v1 on several issues and record the outputs

Input	Model output	Expected
“The application crashes when I log in.”	“This issue reports an error in the application.”	BUG
“Please add dark mode.”	“Feature request”	FEATURE
“How do I install the application?”	“This is a question about installation.”	QUESTION

Evaluate: The model understands the issues, but the outputs do not satisfy the task requirements

- It produces descriptions or alternative terms instead of the required labels

- **Prompt v1** does not specify which labels must be used or what exactly should be returned

Prompt engineering cycle: Example (3) — Refine & Re-test

Refine — Prompt v2: Address the problems identified during evaluation

Task: Classify the issue as exactly one of: **BUG, FEATURE, QUESTION**
Output: Return only the label.
Issue: <issue>{issue}</issue>

Re-test: Run Prompt v2 on the same issues
- "The application crashes when I log in." → BUG
- "Please add dark mode." → FEATURE
- "How do I install the application?" → QUESTION

Evaluate again: the outputs now satisfy the requirements for these test cases
- One of the three allowed labels is returned; only the label is returned
- If other test cases still fail, refine the prompt again and repeat the cycle

Activity 1 — Diagnose and Improve a Prompt

Task: An application uses an LLM to reply to a student asking:
"I missed the assignment deadline yesterday. Can I still submit?"
Note: No late-submission policy or penalty information is provided to the model as part of this application.

Prompt v1
Respond to the student's message.
Student message: {message}

Model output
- **Hypothetical model output** *(you can also try this prompt with an LLM and compare its response)*
- "Yes. You can submit the assignment late. A 10% penalty will apply."

Questions
- What is wrong with this result?
- Why did the prompt allow this to happen?
- How would you improve the prompt?
- How would you test whether your revised prompt works reliably?

Activity 1 — Possible Solution (1)

What is wrong?

- The response invents a late-submission rule and a 10% penalty that were not provided in the prompt.

Why?

- The prompt asks the model to answer the question but provides no information about the actual submission policy.

How to improve it?

- Provide the module policy needed to answer the question
- Tell the model to use only the provided policy
- Tell the model not to invent missing rules or information

Activity 1 — Possible Solution (2)

Test & Evaluate

Test Prompt v2 on several student questions to check whether it consistently:

- Answers using only information in the provided policy
- Does not invent rules or penalties
- Does not provide an answer when it cannot be supported by the provided policy

Part 2: Prompting Techniques

Prompt techniques

- **Zero-shot prompting**: describe the task and ask the model to perform it directly — **no examples are provided**
- **Few-shot prompting / in-context learning**: provide a few examples to **show the model how the task should be performed**
- **Chain-of-Thought**: encourage the model to **work through intermediate reasoning** before producing the final answer
- **ReAct**: combine **reasoning and actions** — reason about what to do, use a tool, observe the result, and continue as needed

Zero-shot prompts

- The prompt tells the model **what to do using instructions alone**, rather than showing examples of the expected input and output

- Works well when the task is **familiar and can be clearly specified**
  - e.g., translate a sentence, summarise a document

- May be insufficient when instructions alone do not clearly communicate the expected behaviour or output; examples can make the expectation more concrete, for example:

  - **Custom categories**: classify customer messages as **Complaint, Request or Enquiry**, when the organisation has its **own rules for deciding which category applies**

  - **Specific output style**: write product descriptions “in our usual style” when that style has not been explained

Few-shot prompting / In-context learning

- When describing the task is not enough to communicate exactly what is expected, we can **show the model examples**

- **Few-shot prompting / in-context learning (ICL):** provide examples of how the task should be performed as part of the prompt

- Provide **input/output pairs**, followed by the new input
  - **One-shot:** one input/output example
  - **Few-shot:** several input/output examples

- **The model does not learn these examples permanently**: they only guide the model for the current prompt — the model itself is not changed (unlike fine-tuning)

- Examples should be **representative of the cases the model needs to handle**
  - e.g., for review classification: include examples covering **positive, negative, mixed and neutral reviews**

Few-shot Prompting: Example

Task: Classify a product review as Positive, Negative or Neutral

Examples

Review: “I love these headphones. The sound quality is excellent.”
Sentiment: Positive

Review: “The product is fine, but nothing stands out.”
Sentiment: Neutral

Review: “It stopped working after two days.”
Sentiment: Negative

New input

Review: “The product works fine, but delivery was slow.”
Sentiment: ?

The examples show the model the task, the allowed categories and the expected output format.

Chain-of-thought (CoT) prompting

- Instead of asking the model to produce an answer **directly**, CoT encourages it to **work through intermediate reasoning steps before reaching the final answer**
- The intermediate steps **break a complex problem into smaller reasoning steps**, allowing the model to work through the problem progressively
- Particularly useful when solving a task requires **combining several pieces of reasoning in sequence**, rather than reaching the answer in a single step
- CoT can be prompted by **providing examples that show the reasoning steps and final answer**, or by explicitly asking the model to **reason step by step**
- CoT can improve reasoning performance, but **a plausible chain of reasoning does not guarantee a correct answer**

Ref: J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, 2022

Chain-of-Thought: Example

Prompt:

"CA has a **weight of 40%** and the exam has a **weight of 60%** in the final module mark. A student received a **CA mark of 65 out of 100** and needs a **final module mark of 50 out of 100** to pass.

What exam mark does the student need to pass? Work through the calculation step by step before giving the final answer."

Model response:

- CA contribution: $65 \times 0.40 =$ **26 marks**
- Marks still needed: $50 - 26 =$ **24 marks**
- Required exam mark: $24 \div 0.60 =$ **40 marks**
- **Final answer: 40 marks out of 100**

The prompt explicitly asks the model to **reason through the intermediate steps before giving the final answer**.

When is Chain-of-Thought Useful?

- CoT is most useful for tasks that require **multiple dependent reasoning steps**
- Research shows that its benefits are strongest for **mathematical and symbolic reasoning tasks**, rather than across all types of tasks
- CoT is **not always needed**: tasks that can be answered directly may not benefit from generating intermediate reasoning steps
- CoT also has a **cost**: generating intermediate reasoning uses more tokens and can increase response time
- Therefore, use CoT **selectively**, when the task requires multi-step reasoning, and evaluate whether it improves the results

- **Ref:** Z. Sprague et al., "To CoT or Not to CoT? Chain-of-Thought Helps Mainly on Math and Symbolic Reasoning," *ICLR*, 2025.

ReAct (Reason + Act)

- **ReAct (Reason + Act):** the model alternates between **reasoning about what to do next** and **using tools** to obtain information or perform an action

- **ReAct** follows a repeating cycle:
  - **Thought:** determine what is needed next
  - **Action:** use a tool
  - **Observation:** receive the result of the action
  - then **reason again** using the new information

- The cycle continues until the model has enough information to **complete the task or provide the final answer**

- ReAct is an important foundation for **LLM agents that reason and use tools iteratively to complete tasks**

Ref: S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023.

ReAct : Example

Prompt:

Follow this process: **Thought → Action → Observation**. Repeat as needed, then provide the **Final Answer**

Task: "Find Jane Smith's most recent paper on LLM agents and identify her institution."

- **Thought:** I need to find her most recent paper on LLM agents and where it was published.
- **Action:** Search the research-paper database.
- **Observation:** "Adaptive LLM Agents," AAMAS 2026.
- **Thought:** I need the affiliation reported for this paper.
- **Action:** Retrieve the paper details.
- **Observation:** Jane Smith — University of Edinburgh.
- **Final Answer:** "Adaptive LLM Agents," AAMAS 2026 — University of Edinburgh.

CoT vs ReAct: When to Use Which?

- **CoT:** use when the model needs to **work through several reasoning steps to reach an answer**, without needing external tools
  - e.g., calculate the exam mark needed to pass from the marks and weights provided

- **ReAct:** use when the model needs to **reason and use external tools** to complete the task
  - e.g., check the **current price of a product** and determine whether it is within a given budget

- **ReAct** can handle tasks that require **external information or actions**, but using tools adds **cost and additional ways the task can fail**

- Therefore, use **ReAct** when **tool interaction is needed as part of the reasoning process**, rather than for every reasoning task

Activity 2A — Choose the Prompting Technique

Task: Extract information

Given a student email, return:

- Student name
- Module code
- Requested action

Use this format:

- Student name: ..., Module code: ..., Requested action: ...

Questions

- Which prompting technique would you use?
- Why?

Activity 2A: Possible Solution

Technique: Zero-shot prompting

The task can be specified directly without providing examples.

The prompt already tells the model:

- What information to extract
- Which fields to return
- How to format the response

Therefore, examples are not necessary unless testing shows that the zero-shot prompt does not work reliably.

Activity 2B — Choose the Prompting Technique

Task: Classify IT requests

A university classifies IT requests as:

- Access
- Hardware
- Software
- Network

Some requests could fit more than one category. The university has examples showing how such cases should be classified.

Questions

- Which prompting technique would you use?
- Why?

Activity 2B: Possible Solution

Technique: Few-shot prompting

The university has its own rules for how requests should be classified. Examples can show the model how these rules should be applied.

Example demonstrations

- “Wi-Fi keeps disconnecting in the library.” → Network
- “I cannot log in to my university account.” → Access
- “Microsoft Word will not open on my laptop.” → Software
- “My university laptop screen is not working.” → Hardware

The examples demonstrate what each category means in this specific task.

Activity 2C — Choose the Prompting Technique

Task — Plan conference travel within a budget

You have a travel budget of €1,200.

- Conference registration: €450
- Flight: €280
- Hotel: €135 per night for 3 nights
- Local transport: €40

Determine whether the budget is sufficient and, if so, how much remains.

Questions

- Which prompting technique would you use?
- Why?

Activity 2C: Possible Solution

Technique: Chain-of-Thought (CoT)

The task requires several dependent calculations:

- Hotel cost: €135 × 3 = €405
- Total cost: €450 + €280 + €405 + €40 = €1,175
- Remaining budget: €1,200 − €1,175 = €25

Answer: The budget is sufficient, with €25 remaining.

CoT is appropriate because the answer requires working through multiple calculation steps before reaching the final result.

Activity 2D — Choose the Prompting Technique

Task — Check whether a book is currently available

A student wants to borrow a specific book from the university library.

The model needs to:

- Find the book in the library catalogue
- Check its current availability
- If unavailable, check whether another copy is available at a different campus library
- Provide the student with the result

Questions

- Which prompting technique would you use?
- Why?

Activity 2D: Possible Solution

Technique: ReAct

- The task requires the model to **reason and interact with external tools**:
  - Search the library catalogue for the book
  - Observe the search result
  - Check its current availability
  - If unavailable, search other campus libraries
  - Use the results to provide the final answer

ReAct is appropriate because the model must **reason → act → observe → reason again**, rather than answer only from information already provided in the prompt.

Part 3: System Prompt vs User Prompt

System prompt Vs User prompt

- **System prompt**
  - Higher-level instructions that guide the model's behaviour
  - Supplied to the model by the **application**
  - Can define role, rules, constraints and response style

- **User prompt**
  - The instruction/question supplied by the **user**
  - Defines what the user wants the model to do

System Prompt vs User Prompt — Developer Examples

Developer calling Claude API from an application

```
message = client.messages.create(
    model="claude-...",
    system="You are a teaching assistant.",
    messages=[
        {"role": "user",
         "content": "Explain RAG."}
    ]
)
```

Developer calling Gemini API from an application

```
response = client.models.generate_content(
    model="gemini-...",
    contents="Explain RAG.",
    config=types.GenerateContentConfig(
        system_instruction=
            "You are a teaching assistant."
    )
)
```

System prompt supplied by the application. It may be written in code, loaded from configuration, or constructed dynamically.

What Can a System Prompt Specify?

System prompts provide **higher-level instructions that guide the model's behaviour across user requests**. They can specify:

- **Role / persona:** who the assistant is
- **Overall goal:** what the assistant is generally expected to achieve
- **General instructions:** actions or behaviours the assistant should follow across user requests
- **General constraints / boundaries:** limits on those actions or behaviours — what the assistant must not do or conditions it must respect
- **Audience / tone / style:** general expectations for how responses should be written
- **Output conventions:** response structure to use when applicable
- **Tool-use instructions:** which tools can/should be used and under what conditions

These are not exclusive to system prompts. Task-specific instructions, constraints, style or output requirements may also be provided in a user prompt for the current request.

Example: System Prompt

- **Role / persona**
  You are a customer support agent for an online bookstore.
- **Overall goal**
  Help customers resolve problems with their orders.
- **General instructions / rules**
  Verify the order status before giving order-specific advice. Ask for confirmation before cancelling an order.
- **General constraints / boundaries**
  Do not cancel an order that has already been shipped. Do not invent order information.
- **Audience / tone / style**
  Use clear, concise, customer-friendly language.
- **Output format**
  For order enquiries, give the current status first, followed by the recommended next action.
- **Tool-use instructions**
  Use the order lookup tool to retrieve the current order status. Use the cancellation tool only after the customer confirms cancellation.

Activity 3 — Design the System and User Prompts

You are developing an LLM assistant for a **university IT help desk**.

The assistant should:

- Help users resolve university IT problems
- Provide concise, step-by-step guidance
- Not invent account or system information
- Ask for necessary information when it is missing

User prompt:

- “My university Wi-Fi stopped working after I changed my password. What should I do?”

Task

Write:

- the **system prompt**

Activity 3: Possible Solution

System prompt

- **Role / persona**
  You are a university IT help-desk assistant.

- **Overall goal**
  Help users resolve university IT problems.

- **General instructions / rules**
  Ask for necessary information when it is missing.

- **General constraints / boundaries**
  Do not invent account or system information.

- **Audience / tone / style**
  Provide concise, step-by-step guidance.

Structuring prompts

Structuring Prompts

As prompts become more complex, **clear structure helps distinguish instructions from the information they apply to.**

- Clearly distinguish **task/instructions** from the content they apply to
- Use **delimiters or labelled sections** to separate different parts of the prompt *(e.g., **"""..."""**, **<input>...</input>**, **Input:**, **Example:)***

For simple prompts, explicit structure may not be necessary.

Example — Structuring a Prompt

- **Without explicit structure**

Summarize the research paper below in three bullet points, focusing on its main contribution and findings.
[paper text]

- **With explicit structure**

Summarize the research paper below in three bullet points, focusing on its main contribution and findings.
Paper:
"""
[paper text]
"""

- **The task is unchanged. The delimiters make clear where the content to be processed begins and ends.**

Activity 4: Structure the Prompt

An application receives a support ticket and asks an LLM to extract the problem and the requested action.

Current prompt:

Read the support ticket below and identify the problem and requested action.

A user says the printer on the second floor has stopped working and asks for someone to repair it.

Task

- Rewrite the prompt so that the instructions are clearly separated from the support-ticket content. Use appropriate labels or delimiters.

Activity 4: Possible Solution

Revised prompt

Task: Identify the problem and the requested action in the support ticket.

Support ticket:
"""
A user says the printer on the second floor has stopped working and asks for someone to repair it.
"""

What was added to structure the prompt?

- **Task:** clearly separates the instruction from the content.
- **Support ticket:** labels the content the instruction applies to.
- **""" ... """** marks exactly where the support-ticket content begins and ends.

The task itself has **not changed**; only the structure of the prompt has been made explicit.